
Área Científica	Programação de Sistemas de Informação
Unidade Curricular	Introdução às Linguagens e Tecnologias Web
Ano Letivo / Semestre	2025/2026 – 2º Semestre
Docente Responsável	Prof. ^a Catarina Silva
Documento	Apontamentos Teóricos
Tema	<i>Arquitetura Web & Programação Client-Side</i>

1. Arquitetura da World Wide Web e o Paradigma Cliente-Servidor

A World Wide Web constitui um serviço que opera sobre a infraestrutura da Internet, assentando na partilha de documentos interligados através de sistemas de hipertexto. A sua arquitetura basilar rege-se pelo modelo Cliente-Servidor. Neste paradigma, o "Cliente" designa a entidade responsável por efetuar o pedido de informação, sendo classicamente representado por um Web Browser (como o Google Chrome ou o Mozilla Firefox). Em contrapartida, o "Servidor" é a entidade que aloja a informação e processa as respostas a esses pedidos, encontrando-se tipicamente alocado num *datacenter*. Em termos analógicos, este fluxo assemelha-se à dinâmica de um restaurante: o cliente (browser) formula o pedido, o qual é transmitido pela rede (funcionando como o empregado de mesa), até chegar à cozinha (o servidor) que o processa e devolve a refeição.

A comunicação entre estas duas entidades é orquestrada pelo protocolo HTTP (*Hypertext Transfer Protocol*). Uma característica arquitetónica fundamental do HTTP é a sua natureza *stateless* (sem estado); isto significa que cada pedido efetuado é processado de forma totalmente independente, não retendo o servidor, por defeito, qualquer memória de transações passadas.

O processo comunicacional desenvolve-se num ciclo rigoroso de Pedido e Resposta (*Request/Response*). Primeiramente, o cliente emite um *HTTP Request*, que o servidor recebe e processa. Concluído o processamento, o servidor emite um *HTTP Response*, que é finalmente recebido e renderizado no ecrã pelo browser do cliente. Um *HTTP Request* é estruturado por um Método (que dita a ação desejada, como ler ou criar), pelo URL (que identifica o recurso), por Cabeçalhos ou *Headers* (que transportam metadados, como o tipo de browser), e por um Corpo ou *Body* (que contém os dados a enviar, estando este ausente em pedidos simples do tipo GET). Os métodos HTTP mais relevantes englobam o GET (para leitura de dados), o POST (para criação de novos dados no servidor), o PUT ou PATCH (para a atualização de registos) e o DELETE (para a eliminação de dados).

Por sua vez, a estrutura de um *HTTP Response* incorpora um Código de Estado (*Status Code*), *Headers* que informam sobre o tipo de conteúdo (por exemplo, HTML ou JSON), e o *Body* que contém o recurso solicitado. Os Códigos de Estado são agrupados de forma semântica: a série 1xx tem caráter informativo; a série 2xx denota sucesso (como o 200 OK); a série 3xx indica redirecionamentos; a série 4xx assinala erros originados pelo

cliente (como o 404 Not Found); e a série 5xx aponta para falhas no processamento por parte do servidor (como o 500 Internal Error).

A resposta e gestão destes pedidos dependem dos *Web Servers*, os quais operam numa dualidade: o hardware, que constitui o computador físico onde residem os ficheiros, e o software (como Apache, Nginx ou IIS), que tem a capacidade de interpretar os URLs e o protocolo HTTP. A entrega de conteúdo pode ser Estática, onde o servidor transmite um ficheiro idêntico para todos os utilizadores exatamente como se encontra no disco, ou Dinâmica, em que o servidor executa linguagens (como PHP ou Node.js) e realiza consultas a bases de dados para gerar o código HTML em tempo real antes de o expedir.

2. Fundamentos Semânticos da Programação Client-Side

A programação *Client-Side* engloba o conjunto de instruções executadas localmente no computador do utilizador final, mais especificamente dentro do ambiente do Browser. O propósito primário desta camada de desenvolvimento é dotar a interface de interatividade, permitir a validação de formulários e viabilizar a alteração dinâmica da interface gráfica. No contexto do desenvolvimento Front-End, opera um trio tecnológico standard: o HTML, responsável pela definição da estrutura semântica; o CSS, encarregue da camada de apresentação visual; e a linguagem standard JavaScript (JS), que instaura o comportamento da aplicação.

A integração de JavaScript rege-se por boas práticas que privilegiam a utilização de ficheiros externos incluídos no documento HTML através da tag `<script src="script.js"></script>`. Em termos de sintaxe basal, o JavaScript é uma linguagem *Case Sensitive* (onde a capitalização das letras altera o significado dos identificadores) e utiliza o ponto e vírgula (;) para demarcar o término de uma instrução. Para efeitos de depuração (*debugging*) e testes, os browsers modernos disponibilizam uma Consola de desenvolvimento (frequentemente acessível via F12), a qual permite a impressão de mensagens mediante a instrução `console.log()`.

No domínio da gestão de memória, o JavaScript recorre a variáveis, que atuam como espaços de armazenamento identificados por nomes e contendo valores. A declaração de variáveis no paradigma moderno efetua-se com `let` (para variáveis sujeitas a mutação de valor) ou `const` (para constantes que não permitem reatribuição), devendo a forma obsoleta `var` ser evitada. A nomenclatura destas variáveis obedece a regras estritas, suportando letras, números, *underscores* ou cifrões, não podendo iniciar-se com algarismos.

Os tipos de dados informam o interpretador sobre a natureza da informação armazenada, subdividindo-se em Tipos Primitivos e Objetos. Focando nos Tipos Primitivos, destacam-se a *String* (representação textual delimitada por aspas), o *Number* (que engloba todos os valores numéricos, inteiros ou decimais) e o *Boolean* (entidade de lógica binária assumindo `true` ou `false`, imprescindível para estruturas condicionais). Adicionalmente, existem os tipos *undefined* (variável declarada, mas desprovida de valor atribuído) e *null* (representando a ausência intencional e explícita de um valor).

É imperativo salientar a ocorrência de coerção de tipos em tempo de execução, em que o JavaScript tenta converter tipos automaticamente. Esta inferência pode despoletar anomalias lógicas, como demonstrado na adição do número 1 com a string "1", cujo resultado é a string "11". Para mitigar esta idiosincrasia, é fortemente recomendado o uso do operador de igualdade estrita (`===`), que avalia simultaneamente o valor e o tipo de dado. As operações matemáticas e lógicas são executadas através de operadores aritméticos (adição, subtração, multiplicação, divisão e módulo), de atribuição, de comparação

(maior, menor, igualdade estrita) e operadores lógicos (AND `&&`, OR `||` e o inversor NOT `!`).

3. Estruturas de Controlo, Modularidade e a Manipulação do DOM

A execução natural de um algoritmo decorre de forma sequencial (de cima para baixo). No entanto, as estruturas de controlo de fluxo permitem subverter esta linearidade. As condicionais, representadas pelas instruções `if`, `else if` e `else`, permitem a execução isolada de blocos de código mediante a validação de uma condição verdadeira. Como alternativa elegante para a avaliação de uma única variável com múltiplos valores discretos fixos, emprega-se a instrução `switch`. Por sua vez, as estruturas de repetição (*Ciclos* ou *Loops*) consagram o princípio da engenharia de software DRY (*Don't Repeat Yourself*). O ciclo `for` é implementado quando o número exato de iterações é previamente conhecido, enquanto o ciclo `while` repete a execução de um bloco enquanto uma condição prevalecer verdadeira, exigindo cautela para evitar ciclos infinitos (*Infinite Loops*) que comprometam a aplicação.

A modularidade e o encapsulamento lógico são garantidos pelas Funções. Uma função consubstancia-se num bloco de código desenhado para concretizar uma tarefa específica, passível de ser reutilizada e que apenas entra em execução quando explicitamente invocada no código.

O expoente máximo da programação web cliente reside na interação arquitetada através do DOM (*Document Object Model*). Quando o browser efetua a análise lexical e sintática (*parsing*) do ficheiro HTML, converte as tags numa estrutura de dados hierárquica na memória RAM, sob a forma de uma árvore (DOM Tree). O documento como um todo é instanciado no objeto global `document`, transmutando-se cada tag HTML (como `<html>`, `<body>` ou `<p>`) num Nó (*Node*) ou Objeto pertencente a essa árvore.

A intervenção do JavaScript decorre predominantemente em três fases metódicas. O JavaScript necessita de localizar os nós no DOM através dos métodos de pesquisa do objeto `document`, tais como `getElementById()` ou `querySelector()`. Após a captura da referência do elemento numa variável, é possível alterar o seu conteúdo interno (via propriedade `innerHTML`) ou injetar alterações de estilo de forma programática (via propriedade `style`). A verdadeira interatividade manifesta-se quando o interpretador é instruído para reagir a ações assimétricas perpetradas pelo utilizador (como cliques ou submissões de teclado) acopladas aos elementos mediante métodos como `addEventListener`.

A justificação elementar para a utilização do JavaScript recai sobre a otimização de Tempo e a concretização de Interatividade. As linguagens de marcação (HTML e CSS) estabelecem de forma imperativa o estado inicial da página (o "Ponto A"). O JavaScript projeta a aplicação para o seu estado futuro ("Ponto B"), processando a lógica após o carregamento sem a necessidade de solicitar incessantemente documentos HTML integralmente novos ao servidor perante interações triviais (como a validação de força de uma *password*). Deste modo, assegura-se rapidez operacional com atualização instantânea de fragmentos do ecrã e a ausência de recarregamentos que degradariam a usabilidade.